Summary of Week 2 reading on

Ribeiro, Marco Tulio, Sameer Singh, and Carlos Guestrin. "Why Should I Trust You? Explaining the Predictions of Any Classifier." Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining. 2016.

1. There are two important questions that the paper tries to answer: (a) how should we trust a single model prediction?, and (b) how should we trust the entire model? To answer the first question, they propose a method called Local Interpretable Model-agnostic Explanations (LIME). For the second, they proposed Submodular Pickup LIME (SP-LIME).
2. The approach (a) in 1. can also be thought of as building a local fidelity (≡ faithfulness) model, whereas the approach (b) in 1. can be thought of as a global fidelity model.
3. It seems you start in the original feature space (z), then create an embedding in the transformed space (x). Here, x is the binary representation of z. We have not quite understood how the embeddings z are constructed. Then we perturb x to make a different x', where some 1s are converted to 0s. Note that 0s cannot be converted to 1s (Kaustava suggested that it is perhaps due to the fact that 0 indicates the absence of a variable important and it indicates no reason to suddenly make it non-important. For example, if someone does not have heart disease, you do not want to induce heart disease in the patient artificially.
4. It seems that the perturbations need not be in the original dataset, just that our black-box model should be able to predict on these perturbations. This means that we do not need the ground truth for these perturbed data points.
5. Our understanding is that the fewer the number of features, the more interpretable our model will be.
6. Note that LIME optimizes the following, where f is the black-box ML model output, g is LIME model, and $\pi_x(z)$ use is a proximity measure between an instance z to x, so as to define locality around x.

locality around x. Finally, let $\mathcal{L}(f, g, \pi_x)$ be a measure of how unfaithful g is in approximating f in the locality defined by π_x. In order to ensure both **interpretability** and **local fidelity**, we must minimize $\mathcal{L}(f, g, \pi_x)$ while having $\Omega(g)$ be low enough to be interpretable by humans. The explanation produced by **LIME** is obtained by the following:

$$\xi(x) = \underset{g \in G}{\operatorname{argmin}} \quad \mathcal{L}(f, g, \pi_x) + \Omega(g) \tag{1}$$

This formulation can be used with different explanation families G, fidelity functions $\mathcal{L}$, and complexity measures Ω. Here we focus on sparse linear models as explanations, and on performing the search using perturbations.

7. We also learned that no matter how complicated and non-linear the true boundary is, when you zoom in more and more, you will be able to arrive at a linear boundary.
8. We used $\pi_x(z) = \exp(-\mathcal{D}(x,z)^2/\sigma^2)$ is an exponential kernel defined on some distance function $\mathcal{D}$. The loss is then given by the following equation

$$\mathcal{L}(f, g, \pi_x) = \sum_{z,z' \in \mathcal{Z}} \pi_x(z) \left(f(z) - g(z')\right)^2$$

   Note that the $\pi_x(z)$ is giving less weight to the points which have more distance, implying that if the perturbed instance is very far from the original instanc,e then that will contribute less in loss and that the difference in two predictions of f and g will be downweighted.
9. The *interpretable representation* is explained by using $\Omega(g) = \infty\,\mathbb{I}\big[\|w_g\|_0 > K\big]$, where K is the limit on the number of features (already known). Here, $\|w_g\|_0$ represents the number of non-zero weights. The $\mathbb{I}\big[\|w_g\|_0 > K\big]$ indicator function will be 1 if the number of non-zero weights is more than K, and 0 otherwise. Furthermore, $\infty\mathbb{I}$ notation indicate that $\Omega(g)$ will be ∞ if the number of non-zero weights is more than K, and 0 otherwise.
10. Algorithm 1 appears to do no optimization, yet we know we need to minimize the loss function. So, the minimization is happening in the K-Lasso step, which seems to minimize the loss given the class of at most K features.

Algorithm 1 Sparse Linear Explanations using LIME

Require: Classifier f, Number of samples N
Require: Instance x, and its interpretable version x'
Require: Similarity kernel π_x, Length of explanation K
  $\mathcal{Z} \leftarrow \{\}$
  for $i \in \{1, 2, 3, ..., N\}$ **do**
    $z_i' \leftarrow sample_around(x')$
    $\mathcal{Z} \leftarrow \mathcal{Z} \cup \langle z_i', f(z_i), \pi_x(z_i)\rangle$
  end for
  $w \leftarrow$ K-Lasso$(\mathcal{Z}, K)$ ▷ with z_i' as features, $f(z)$ as target
  return w

11. The authors use K-Lasso as an explanation model, which essentially restricts the class of important variables to at most K.
12. We define a coverage function. The coverage is defined as the set function c that, given $\mathcal{W}$ and $\mathcal{I}$, computes the total importance of the features that appear in at least one instance in a set V. Here $\mathcal{W}$ and $\mathcal{I}$ are as deined in Algorithm 2. V is a set that gets created starting from the empty set of all those instances which maximize the coverage function given $\mathcal{W}$ and $\mathcal{I}$.

$$c(V, \mathcal{W}, I) = \sum_{j=1}^{d'} \mathbb{1}_{[\exists i \in V : \mathcal{W}_{ij} > 0]} I_j$$

Algorithm 2 Submodular pick (SP) algorithm

Require: Instances X, Budget B
  for all $x_i \in X$ **do**
    $\mathcal{W}_i \leftarrow$ **explain**(x_i, x_i') ▷ Using Algorithm 1
  end for
  for $j \in \{1 \dots d'\}$ **do**
    $I_j \leftarrow \sqrt{\sum_{i=1}^{n} |\mathcal{W}_{ij}|}$ ▷ Compute feature importances
  end for
  $V \leftarrow \{\}$
  while $|V| < B$ **do** ▷ Greedy optimization of Eq (4)
    $V \leftarrow V \cup \text{argmax}_i \, c(V \cup \{i\}, \mathcal{W}, I)$
  end while
  return V

13. Next, we try to learn about SP-LIME, which is given in Algorithm 2. Algorithm 2 is clear except for the last part, aka the coverage aspect. Below are the detailed steps of the coverage aspect of Algorithm 2.

    - V is empty to begin with.
    - For the first iteration, we run the loop over all instances and, for each i, calculate the ith row. For each j such that $W_{ij} > 0$, we sum the corresponding I_j's. We add that instance to V, which has the highest non-zero sum of I_j's.
    - In the next step, we check the i_1th row selected in the first iteration and each of the other j instances; the indicator function is 1 for those unique js that have 1s in the two selected rows. Then the coverage function sums the importances across the unique js, and the instance with the maximum coverage is added.
    - This way we proceed until we run out of the budget B.

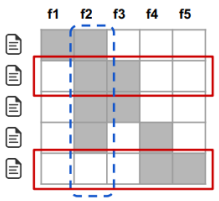

Figure 5: Toy example $\mathcal{W}$. Rows represent instances (documents) and columns represent features (words). Feature f2 (dotted blue) has the highest importance. Rows 2 and 5 (in red) would be selected by the pick procedure, covering all but feature f1.

14. In Figure 5, we first add instance 2. Then in the next step, since likely $I_2 + I_3 + I_4 + I_5$ is very large, then instance 5 is added. It may very well be possible that $I_1 > I_4 + I_5$. If this happens, then instance 1 will be added even though, physically, it does not minimize the coverage. So the coverage function seems to minimize the weighted coverage, where the weights are given by the importances. One feature with high importance is more important than the two features with relatively smaller importances.
15. By the way, note that Algorithm 2 uses W_{ij} (and not w_{ij}. And here, $W_{ij} = |w_{ij}|$. Therefore, W_{ij} is always positive, and then we can just add the values. However, note that the coefficients in regression can be both positive or negative, implying that the presence of the features may result in both negative or positive attributes.
16. A suggestion came up to see if we can cluster in the feature space to begin with, so that we have a better clue of the boundary. But now, I suppose, unsupervised learning may really not help with supervised learning, so this will probably not work.